# Relatório de Evolução — ChargeGridIA

***Sprint 03 — EV Challenge GoodWe (FIAP)***

# 1. Resumo da evolução (Sprints 1 e 2 → Sprint 03)

Nas Sprints 1 e 2, o grupo desenvolveu o chatbot ChargeGridIA para o EV Challenge GoodWe: um assistente de RAG (Retrieval-Augmented Generation) construído com LangChain, usando o modelo **gpt-4o-mini** e um índice vetorial em memória (InMemoryVectorStore) sobre PDFs do carregador GoodWe HCA G2, documentos do desafio e um FAQ complementar. A Sprint 2 já havia montado um pipeline LangGraph simples (retrieve → generate), mas ele só era usado nos testes isolados do notebook — o chatbot interativo de verdade chamava a API da OpenAI diretamente, mantendo o histórico da conversa em uma lista Python cortada manualmente a cada 21 mensagens.

Na Sprint 03, o núcleo conversacional foi refatorado para usar efetivamente o framework de agentes: existe agora um único grafo LangGraph que orquestra toda a conversa, com memória por sessão gerenciada nativamente por um checkpointer (em vez do corte manual de lista) e guardrails de segurança implementados como nós determinísticos do grafo, além do reforço via system prompt já existente. Também foi criado um harness de comparação entre modelos, executando o mesmo conjunto de testes em diferentes LLMs para embasar a escolha do modelo final com dados, não com preferência do grupo.

# 2. Refatoração

## 2.1 Principais decisões técnicas

- **Framework escolhido: LangGraph.** O grupo já usava LangChain/LangGraph parcialmente na Sprint 2 (pipeline retrieve→generate), então a Sprint 03 aproveitou essa base em vez de trocar de família de ferramentas — o esforço de refatoração foi concentrado em unificar o grafo, e não em reaprender um framework novo do zero.
- **Memória de sessão via checkpointer (MemorySaver), chaveada por `thread_id`.** Substitui o corte manual de histórico da Sprint 2. Cada sessão de usuário vira uma thread independente; o framework decide como persistir e recuperar as mensagens, eliminando a lógica manual de truncamento.
- **Guardrails como nós do grafo.** Um nó `check_input_guardrails` roda antes da geração, detectando tentativas de prompt injection por padrão de texto; um nó `check_output_guardrails` roda depois, sinalizando respostas que pareçam conter aconselhamento jurídico/financeiro indevido. Isso cria uma segunda camada de defesa que não depende só do modelo "lembrar" de seguir o system prompt.
- **RAG mantido, mas desacoplado do Colab.** A lógica de carregar PDFs/DOCX/FAQ da Sprint 2 foi preservada, mas movida para um módulo local (`src/rag.py`), sem dependência de Google Drive/Colab — atendendo ao requisito de a aplicação permanecer local (seção 10 do enunciado).

## 2.2 Trade-offs identificados

- **Checkpointer em memória (MemorySaver) vs. persistente (SQLite/Postgres):** optou-se por MemorySaver para simplicidade de demonstração; em produção real a sessão se perderia ao reiniciar o processo. O LangGraph oferece checkpointers persistentes como troca direta caso o grupo precise disso.
- **Guardrails por regex vs. por LLM:** a detecção de prompt injection e de escopo usa padrões de texto (regex), que é rápido e determinístico mas não pega paráfrases muito criativas. Um guardrail baseado

em outro LLM seria mais robusto, porém mais lento e mais caro por turno — trade-off explícito entre custo/latência e cobertura.

- **Granularidade do grafo:** o grafo atual tem 5 nós (entrada, recusa, retrieve, generate, saída). Seria possível quebrar ainda mais (ex: nó dedicado só para classificação de escopo), mas o grupo optou por manter o grafo enxuto o suficiente para ser explicável na sustentação.

# 3. Comparativo antes × depois

A tabela abaixo resume a evolução estrutural. As colunas de métrica quantitativa (latência, tokens/turno, nota nos testes) devem ser preenchidas pelo grupo após rodar `python -m tests.test_cases` contra as duas versões, usando o mesmo conjunto de perguntas nas duas rodadas.

| Critério | Sprint 1 e 2 (legado) | Sprint 03 (framework de agentes) |
|---|---|---|
| Arquitetura | Loop manual + chamada direta à API OpenAI | Grafo LangGraph (5 nós) com roteamento condicional |
| Memória | Lista Python cortada manualmente (>21 msgs) | Checkpointer nativo (MemorySaver) por thread_id |
| Guardrails | Somente texto no system prompt | System prompt + nós determinísticos de entrada/saída |
| Seleção de modelo | Fixo em gpt-4o-mini, por preferência | Harness compara N modelos com o mesmo teste |
| Qualidade das respostas | [preencher após rodar testes] | [preencher após rodar testes] |
| Tokens por turno | [preencher] | [preencher] |
| Latência média | [preencher] | [preencher] |
| Resultado testes de segurança | [preencher] | [preencher] |

**A nova arquitetura realmente tornou o chatbot melhor?** Estruturalmente, sim: a memória deixa de depender de lógica manual propensa a erro, os guardrails passam a ser testáveis isoladamente (cada um é uma função pura), e a escolha de modelo passa a ser baseada em experimento. A confirmação quantitativa (linhas em branco da tabela acima) depende de rodar os scripts de teste com as chaves de API reais do grupo.

# 4. Problemas encontrados e soluções

## 4.1 Problema 1 — Memória gerenciada manualmente mascarava bugs de contexto

**Problema:** na Sprint 2, o corte de histórico ("últimas 20 mensagens") era feito à mão dentro do loop principal, misturado com lógica de I/O do terminal. Isso dificultava testar a memória isoladamente e criava risco de cortar uma mensagem de sistema no meio da lista.

**Alternativas possíveis:** (a) manter o corte manual, mas isolado em função própria; (b) usar um checkpointer do LangGraph; (c) implementar um buffer de memória com resumo automático (summarization) para conversas muito longas.

**Solução adotada:** opção (b) — checkpointer nativo do LangGraph (MemorySaver), com sessão identificada por thread_id.

**Justificativa:** remove completamente a lógica manual de corte do código de aplicação, delega a responsabilidade ao framework (que é o próprio objetivo da Sprint 03) e permite testar memória chamando o grafo diretamente, sem depender do loop de terminal.

### 4.2 Problema 2 — Guardrails eram só instrução de prompt, difíceis de testar

**Problema:** as regras de segurança da Sprint 2 (não dar conselho jurídico, não revelar o prompt, etc.) existiam apenas como texto dentro do system prompt. Não havia forma determinística de verificar se uma tentativa de prompt injection foi de fato bloqueada, além de ler a resposta do modelo manualmente.

**Alternativas possíveis:** (a) manter apenas no system prompt e confiar no modelo; (b) adicionar um classificador de segurança baseado em outro LLM antes de cada resposta; (c) adicionar checagens determinísticas (regex/keywords) como nós explícitos do grafo.

**Solução adotada:** opção (c), combinada com o system prompt já existente (defesa em profundidade), com opção de evoluir para (b) em versões futuras.

**Justificativa:** checagens por regex são determinísticas, rápidas e não dependem de uma chamada extra de API — o que mantém a latência baixa — e por serem nós isolados do grafo, podem ser testadas unitariamente sem precisar rodar o LLM inteiro.

## 5. Divisão da equipe

| Nome | RM | Principal responsabilidade |
|---|---|---|
| Bruna Yukimy Hada | 571836 | [preencher] |
| Denize Ferrante | 571562 | [preencher] |
| Gabriel Del Pizzo Pintor | 570436 | [preencher] |
| Gabriel Dias Menezes | 572395 | [preencher] |
| Ian Rodrigues Martins | 570540 | [preencher] |
| Patrick Fernandes Martins Pais | 572899 | [preencher] |

Nota: campos marcados como [preencher] dependem de dados que só existem após a equipe rodar os scripts de teste (tests/test_cases.py e compare_models.py) com suas próprias chaves de API, e após decidir a divisão de responsabilidades de cada integrante.